Health Report

```
-- ============================================================
--  health_report_function.sql
--  PostgreSQL health-report helper “package”
--  ------------------------------------------------------------
--  Creates:
--    • schema health_report
--    • view health_report.health_alerts (aggregates all alerts)
--    • functions:
--        run_health_report()
--        run_health_summary()
--        run_health_critical()
--        has_critical_alerts()
--  All functions return SETOF records so they can be queried
--  directly (e.g. SELECT * FROM health_report.run_health_report();)
-- ============================================================

------------------------------------------------------------
-- 1. Create a dedicated schema (if it does not already exist)
------------------------------------------------------------
CREATE SCHEMA IF NOT EXISTS health_report;
COMMENT ON SCHEMA health_report IS
    'Namespace for health-report utilities used by Lumo.';

------------------------------------------------------------
-- 2. Helper: generic alert collector (used by the view below)
------------------------------------------------------------
-- This CTE builds a unified list of possible alerts.  Each block
-- adds rows with a consistent shape:
--   alert_id   SERIAL,
--   severity   TEXT      -- ''CRITICAL'', ''WARN'', ''INFO''
--   source     TEXT      -- which check produced it
--   description TEXT
--   details    JSONB
------------------------------------------------------------
CREATE OR REPLACE FUNCTION health_report._collect_alerts()
RETURNS TABLE (
    alert_id    BIGINT,
    severity    TEXT,
```

```
    source      TEXT,
    description TEXT,
    details     JSONB
) LANGUAGE plpgsql AS
$$
DECLARE
    _id BIGINT := 0;
BEGIN
    -------------------------------------------------------------
    -- 2.1  Critical: Table bloat > 10%
    -------------------------------------------------------------
    FOR rec IN
        SELECT
            nspname || '.' || relname             AS tbl,
            pg_total_relation_size(c.oid)         AS total_bytes,
            pg_relation_size(c.oid)               AS data_bytes,
            pg_total_relation_size(c.oid) -
                pg_relation_size(c.oid)           AS toast_bytes,
            CASE
                WHEN pg_total_relation_size(c.oid) = 0 THEN 0
                ELSE
                    round(
                        (pg_total_relation_size(c.oid) -
                         pg_relation_size(c.oid)) *
                         100.0 /
                         pg_total_relation_size(c.oid), 2)
            END                                   AS bloat_pct
        FROM pg_class c
        JOIN pg_namespace n ON n.oid = c.relnamespace
        WHERE c.relkind = 'r'                     -- only ordinary tables
          AND n.nspname NOT IN ('pg_catalog','information_schema')
    LOOP
        IF rec.bloat_pct > 10 THEN
            _id := _id + 1;
            RETURN NEXT (
                _id,
                'CRITICAL',
                'table_bloat',
                format('Table %s has %.2f%% bloat', rec.tbl, rec.bloat_pct),
                jsonb_build_object(
                    'table', rec.tbl,
```

```
            'total_bytes', rec.total_bytes,
            'data_bytes', rec.data_bytes,
            'toast_bytes', rec.toast_bytes,
            'bloat_percent', rec.bloat_pct
        )
    );
  END IF;
END LOOP;

-- ------------------------------------------------------------------
-- 2.2  Critical: Replication lag > 30 seconds (if streaming replica)
-- ------------------------------------------------------------------
IF EXISTS (SELECT 1 FROM pg_stat_replication) THEN
  FOR rec IN
    SELECT
      client_addr,
      pg_last_xact_replay_timestamp() AS last_replay,
      now() - pg_last_xact_replay_timestamp() AS lag
    FROM pg_stat_replication
  LOOP
    IF rec.lag > interval '30 seconds' THEN
      _id := _id + 1;
      RETURN NEXT (
        _id,
        'CRITICAL',
        'replication_lag',
        format('Replica %s lagging by %s', rec.client_addr, rec.lag),
        jsonb_build_object(
          'client_addr', rec.client_addr,
          'lag_seconds', extract(epoch FROM rec.lag)
        )
      );
    END IF;
  END LOOP;
END IF;

-- ------------------------------------------------------------------
-- 2.3  Warn: Tables with > 5% dead tuples (vacuum needed)
-- ------------------------------------------------------------------
FOR rec IN
  SELECT
```

```
        schemaname || '.' || relname              AS tbl,
        n_dead_tup,
        n_live_tup,
        CASE
            WHEN (n_dead_tup + n_live_tup) = 0 THEN 0
            ELSE round(n_dead_tup * 100.0 / (n_dead_tup + n_live_tup), 2)
        END                                       AS dead_pct
    FROM pg_stat_user_tables
LOOP
    IF rec.dead_pct > 5 THEN
        _id := _id + 1;
        RETURN NEXT (
            _id,
            'WARN',
            'dead_tuples',
            format('Table %s has %.2f%% dead tuples', rec.tbl, rec.dead_pct),
            jsonb_build_object(
                'table', rec.tbl,
                'dead_tuples', rec.n_dead_tup,
                'live_tuples', rec.n_live_tup,
                'dead_percent', rec.dead_pct
            )
        );
    END IF;
END LOOP;

-- ------------------------------------------------------------------
-- 2.4  Info: Database size
-- ------------------------------------------------------------------
_id := _id + 1;
RETURN NEXT (
    _id,
    'INFO',
    'db_size',
    'Current database size',
    jsonb_build_object(
        'size_bytes', pg_database_size(current_database())
    )
);

-- ------------------------------------------------------------------
```

```
    -- Add additional checks here (e.g., missing backups, connection
    -- limits, long-running queries, etc.) following the same pattern.
    ------------------------------------------------------------------
END;
$$
SECURITY DEFINER;  -- allows non-superusers to run the health checks


----------------------------------------------------------------
-- 3. View that surfaces all alerts in a convenient table
----------------------------------------------------------------
DROP VIEW IF EXISTS health_report.health_alerts CASCADE;
CREATE VIEW health_report.health_alerts AS
SELECT
    alert_id,
    severity,
    source,
    description,
    details
FROM health_report._collect_alerts()
ORDER BY severity DESC, alert_id;

COMMENT ON VIEW health_report.health_alerts IS
    'Unified view of all health alerts (CRITICAL, WARN, INFO).';


----------------------------------------------------------------
-- 4. Function: has_critical_alerts()
----------------------------------------------------------------
CREATE OR REPLACE FUNCTION health_report.has_critical_alerts()
RETURNS boolean
LANGUAGE sql STABLE
AS $$
    SELECT EXISTS (
        SELECT 1
        FROM health_report.health_alerts
        WHERE severity = 'CRITICAL'
    );
$$;

COMMENT ON FUNCTION health_report.has_critical_alerts() IS
```

```
    'True if any CRITICAL alerts are present.';


-------------------------------------------------------------------
-- 5. Function: run_health_summary()
-------------------------------------------------------------------
-- Returns a compact, one-row-per-category summary.
-------------------------------------------------------------------
CREATE OR REPLACE FUNCTION health_report.run_health_summary()
RETURNS TABLE (
    section      text,
    status       text,
    count        bigint
) LANGUAGE plpgsql STABLE AS
$$
BEGIN
    -- 5.1  Number of tables
    RETURN QUERY SELECT
        'tables'      AS section,
        'OK'          AS status,
        count(*)      AS count
    FROM pg_tables
    WHERE schemaname NOT IN ('pg_catalog','information_schema');

    -- 5.2  Number of indexes
    RETURN QUERY SELECT
        'indexes'     AS section,
        'OK'          AS status,
        count(*)      AS count
    FROM pg_indexes
    WHERE schemaname NOT IN ('pg_catalog','information_schema');

    -- 5.3  Total dead tuples (warn if > 0)
    RETURN QUERY SELECT
        'dead_tuples' AS section,
        CASE WHEN sum(n_dead_tup) > 0 THEN 'WARN' ELSE 'OK' END,
        sum(n_dead_tup) AS count
    FROM pg_stat_user_tables;

    -- 5.4  Replication lag status (if applicable)
    IF EXISTS (SELECT 1 FROM pg_stat_replication) THEN
```

```
        RETURN QUERY SELECT
            'replication_lag' AS section,
            CASE
                WHEN max(now() - pg_last_xact_replay_timestamp()) > interval '30 seconds'
                THEN 'WARN' ELSE 'OK'
            END,
            EXTRACT(EPOCH FROM max(now() - pg_last_xact_replay_timestamp()))::bigint
        FROM pg_stat_replication;
    END IF;

    -- 5.5  Critical alerts count
    RETURN QUERY SELECT
        'critical_alerts' AS section,
        CASE WHEN cnt > 0 THEN 'WARN' ELSE 'OK' END,
        cnt
    FROM (
        SELECT count(*) AS cnt
        FROM health_report.health_alerts
        WHERE severity = 'CRITICAL'
    ) sub;

    -- 5.6 Extensions list (optional)
    RETURN QUERY SELECT
        'installed_extensions' AS report_section,
        jsonb_agg(to_jsonb(e))
    FROM health_report.list_installed_extensions() e;
END;
$$;

COMMENT ON FUNCTION health_report.run_health_summary() IS
    'Provides a high-level numeric summary of database health.';



-------------------------------------------------------------
-- 6. Function: run_health_critical()
-------------------------------------------------------------
-- Returns only the critical issues with a short description.
-------------------------------------------------------------
CREATE OR REPLACE FUNCTION health_report.run_health_critical()
RETURNS TABLE (
```

```
    issue   text,
    detail  text
) LANGUAGE sql STABLE AS
$$
    SELECT
        source   AS issue,
        description AS detail
    FROM health_report.health_alerts
    WHERE severity = 'CRITICAL'
    ORDER BY alert_id;
$$;

COMMENT ON FUNCTION health_report.run_health_critical() IS
    'List of all CRITICAL health issues.';



-------------------------------------------------------------
-- 7. Function: run_health_report()
-------------------------------------------------------------
-- Master function that bundles the three detailed helpers into a
-- single JSON-structured report.  Each row contains a section name
-- and a JSON payload with the relevant data.
-------------------------------------------------------------
CREATE OR REPLACE FUNCTION health_report.run_health_report()
RETURNS TABLE (
    report_section text,
    details        jsonb
) LANGUAGE plpgsql STABLE AS
$$
BEGIN
    -- 7.1  Summary
    RETURN QUERY SELECT
        'summary'   AS report_section,
        jsonb_agg(to_jsonb(t)) AS details
    FROM (
        SELECT * FROM health_report.run_health_summary()
    ) t;

    -- 7.2  Critical issues (array of objects)
    RETURN QUERY SELECT
        'critical_issues' AS report_section,
```

```
        jsonb_agg(jsonb_build_object('issue', issue, 'detail', detail))
    FROM health_report.run_health_critical();

    -- 7.3  Full alert list (optional — can be huge)
    RETURN QUERY SELECT
        'all_alerts' AS report_section,
        jsonb_agg(to_jsonb(a))
    FROM health_report.health_alerts a;

    -- 7.4  Raw health_alerts view filtered to CRITICAL/WARN/INFO
    -- (kept separate for callers that prefer tabular output)
    RETURN QUERY SELECT
        'raw_alerts' AS report_section,
        jsonb_agg(to_jsonb(a))
    FROM health_report.health_alerts a;
END;
$$;

COMMENT ON FUNCTION health_report.run_health_report() IS
    'Top-level entry point – returns a JSON-packed health report covering '
    'summary, critical issues and the full alert list.';

— 7.5 check installed extension
-- ------------------------------------------------------------
--  health_report.list_installed_extensions()
--  Returns a set of rows describing every extension that is
--  installed in the current database.
-- ------------------------------------------------------------
CREATE OR REPLACE FUNCTION health_report.list_installed_extensions()
RETURNS TABLE (
    extension_name   text,
    extension_version text,
    installed_at     timestamp with time zone
) LANGUAGE sql STABLE AS
$$
    SELECT
        extname         AS extension_name,
        extversion      AS extension_version,
        pg_extension.extcreated AT TIME ZONE current_setting('TimeZone')
                        AS installed_at
    FROM pg_extension
```

```
    ORDER BY extname;
$$;

COMMENT ON FUNCTION health_report.list_installed_extensions() IS
    'Lists all extensions installed in the current database, together with '
    'their version and installation timestamp.';



-------------------------------------------------------------
-- 8. Grant usage to PUBLIC (adjust as needed for your security model)
-------------------------------------------------------------
GRANT USAGE ON SCHEMA health_report TO PUBLIC;
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA health_report TO PUBLIC;
GRANT SELECT ON ALL TABLES IN SCHEMA health_report TO PUBLIC;

-------------------------------------------------------------
-- 9. OPTIONAL: Install the pgstattuple extension for more precise
--    bloat calculations (requires superuser rights).
-------------------------------------------------------------
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_extension WHERE extname = 'pgstattuple') THEN
        CREATE EXTENSION IF NOT EXISTS pgstattuple;
    END IF;
EXCEPTION
    WHEN insufficient_privilege THEN
        RAISE NOTICE 'pgstattuple extension could not be installed – '
                     'bloat detection will rely on size heuristics.';
END;
$$;

-------------------------------------------------------------
-- End of health_report_function.sql
-------------------------------------------------------------
```

Installl

health_report_function.sql file.
Copy the entire block (including the opening and closing sql lines) into a plain-text editor and

save it with the name health_report_function.sql. You can then load it into PostgreSQL with \i /path/to/health_report_function.sql.

```
SELECT * FROM run_health_report();
SELECT * FROM run_health_summary();
SELECT * FROM run_health_critical();
SELECT * FROM has_critical_alerts();
SELECT * FROM health_alerts where severity = 'CRITICAL';
SELECT * FROM health_report.list_installed_extensions();


-- Combine with the health report
SELECT
    r.report_section,
    r.details
FROM health_report.run_health_report() r
UNION ALL
SELECT
    'installed_extensions' AS report_section,
    jsonb_agg(to_jsonb(e)) AS details
FROM health_report.list_installed_extensions() e;
```